

Programming Assignment 1

Introduction to Computer Networks — Prof. Patt-Shamir

Submission date: 2026-05-20

Submitted by

Name	Student ID
Yuval Horowitz	206587719
Avraham Mazuz	318436573
Yotam Gilboa	313294266

Submission Contents

File	Description
client.c	UDP job-generator. Samples Poisson inter-arrival times and exponential job lengths via the spec's randexp; sends each job as a 10-byte datagram in network byte order; logs a TSV line per job.
server.c	Multi-threaded UDP server. Acceptor (main) thread calls recvfrom and enqueues jobs into a synchronized STAILQ; worker thread dequeues, sleeps for the job's length to simulate processing, and logs the per-job statistics. Drop-tail policy when the FIFO is full. Mutex + condition variable for synchronization; clock_gettime(CLOCK_MONOTONIC) for timing.
Makefile	Build script. `make` compiles both binaries with -Wall -Wextra -O2 -std=c11 -Wpedantic, plus -pthread for the server and -lm for the client. `make clean` removes the binaries.
README.pdf	This document.

Implementation Summary

The system consists of a UDP client that generates jobs according to a Poisson process (parameter λ) with exponentially distributed service-time stamps (parameter μ), and a multi-threaded UDP server that receives jobs into a bounded FIFO and processes them on a dedicated worker thread.

The server's two threads share a STAILQ-backed FIFO protected by a pthread mutex; the worker waits on a condition variable when the queue is empty and is woken by the acceptor on each enqueue. A done flag, set by the acceptor after num_jobs receives, cooperates with the worker to drain remaining jobs and exit cleanly.

The 8th column of the server log (q_time) and the 7th (q_num) include the just-finished job per the lecturer's clarification — counters are decremented only after the log line is printed.

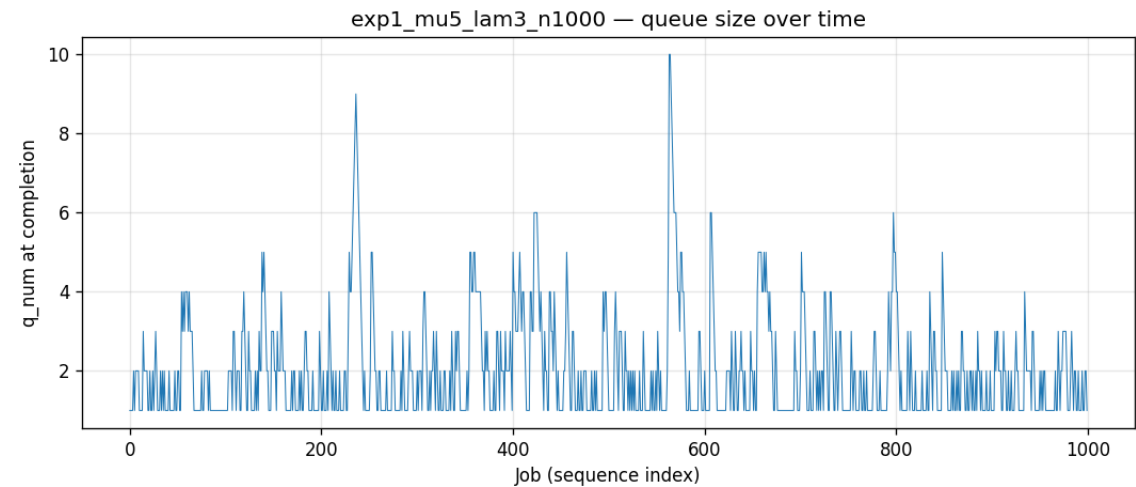
Experimental Results

Experiment 1 — Single client, unbounded queue

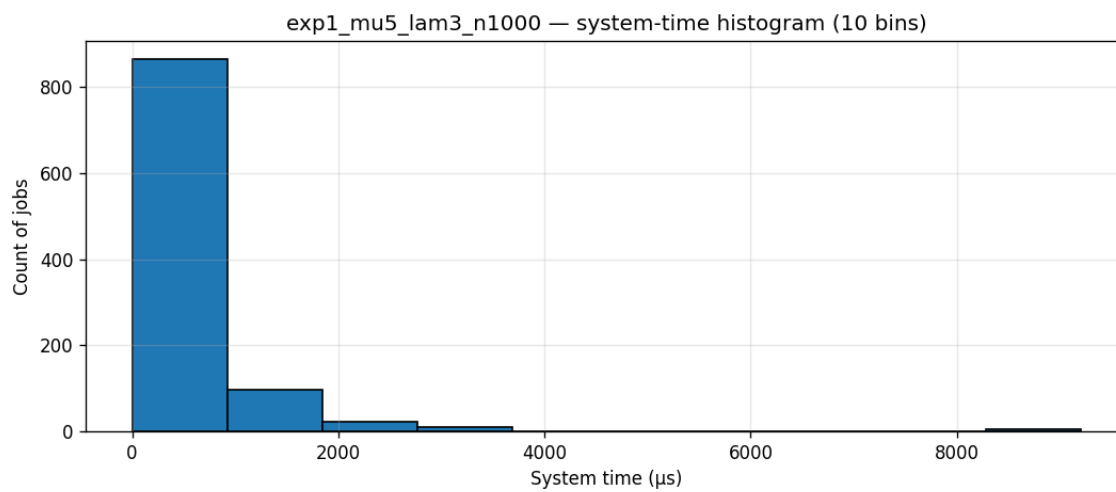
Various (μ , λ) combinations to study queueing behavior across utilization levels.

exp1_mu5_lam3_n1000

μ	5	λ	3
$\rho = \lambda/\mu$	0.600	Sent	1000
Processed	1000	Mean job time (μ s)	533.3
Median job time (μ s)	345.5	Mean q_num	1.98
Median q_num	2.0	Max q_num	10



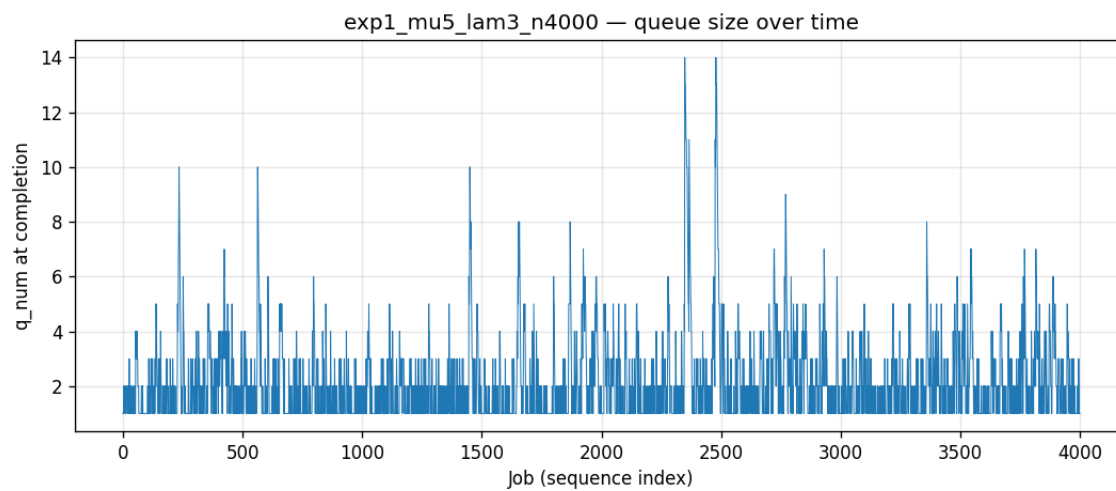
Queue size (q_num) at each job's completion, in chronological order.



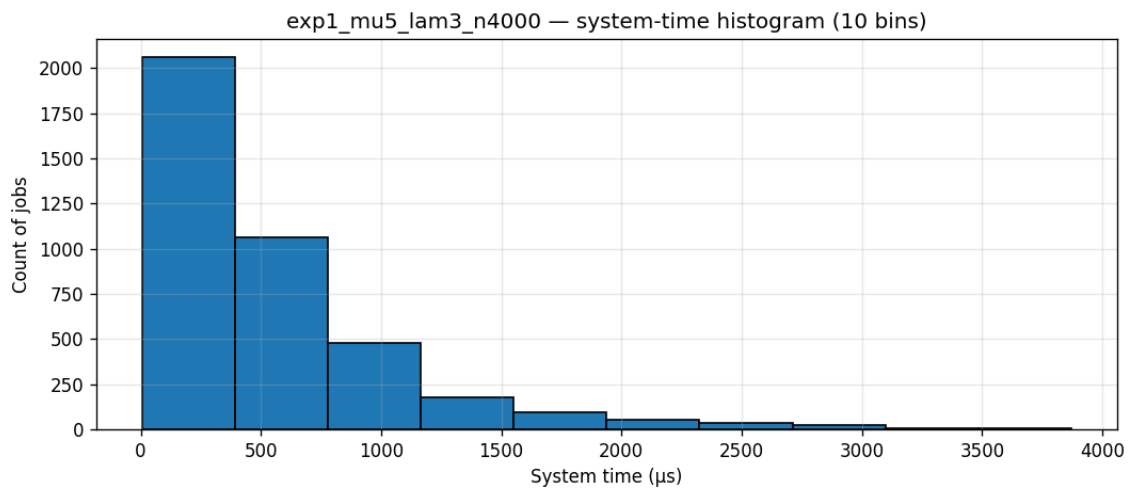
Histogram of per-job system times (departure – arrival), 10 bins.

exp1_mu5_lam3_n4000

μ	5	λ	3
$\rho = \lambda/\mu$	0.600	Sent	4000
Processed	4000	Mean job time (μ s)	536.2
Median job time (μ s)	376.5	Mean q_num	2.10
Median q_num	2.0	Max q_num	14



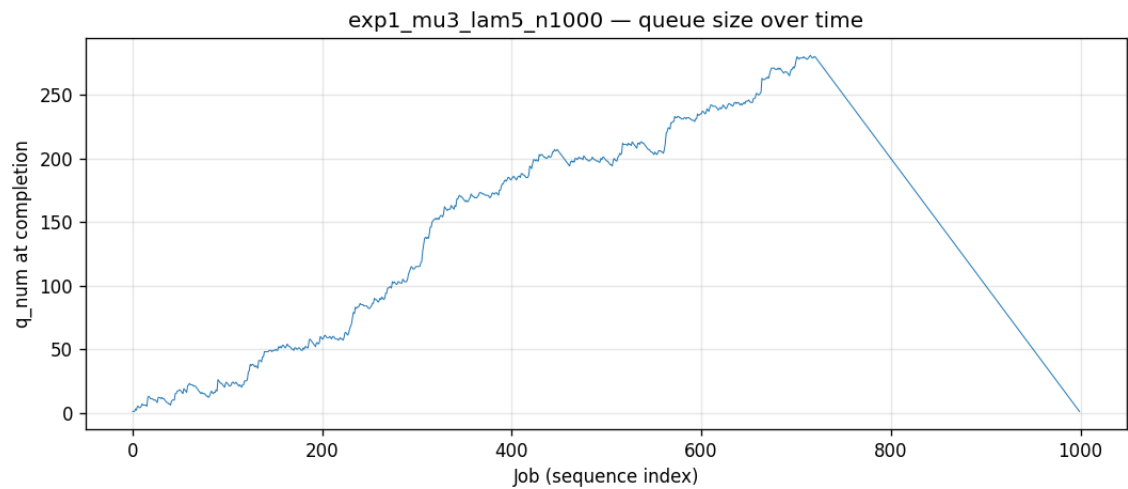
Queue size (q_num) at each job's completion, in chronological order.



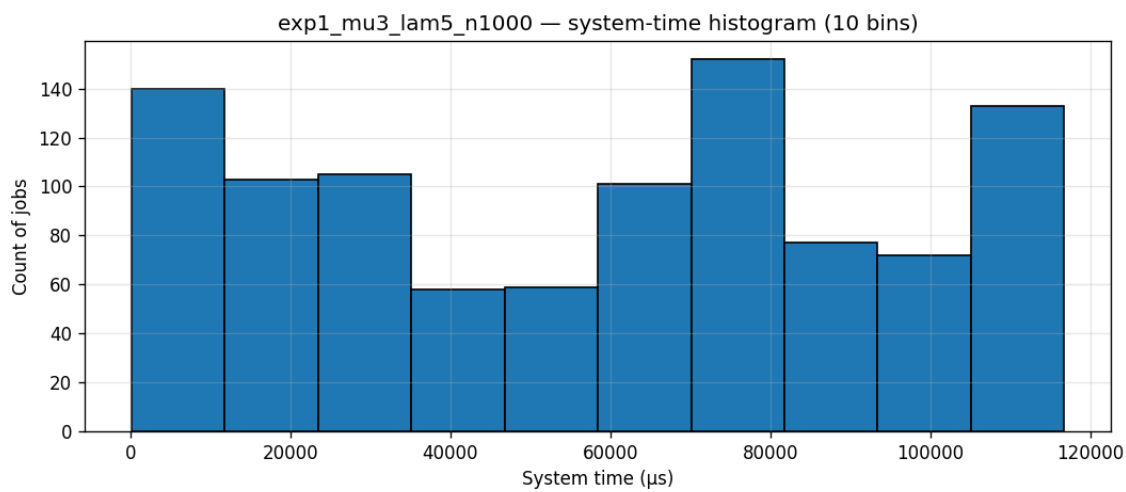
Histogram of per-job system times (departure – arrival), 10 bins.

exp1_mu3_lam5_n1000

μ	3	λ	5
$\rho = \lambda/\mu$	1.667	Sent	1000
Processed	1000	Mean job time (μ s)	57425.8
Median job time (μ s)	61663.0	Mean q_num	141.48
Median q_num	162.0	Max q_num	281



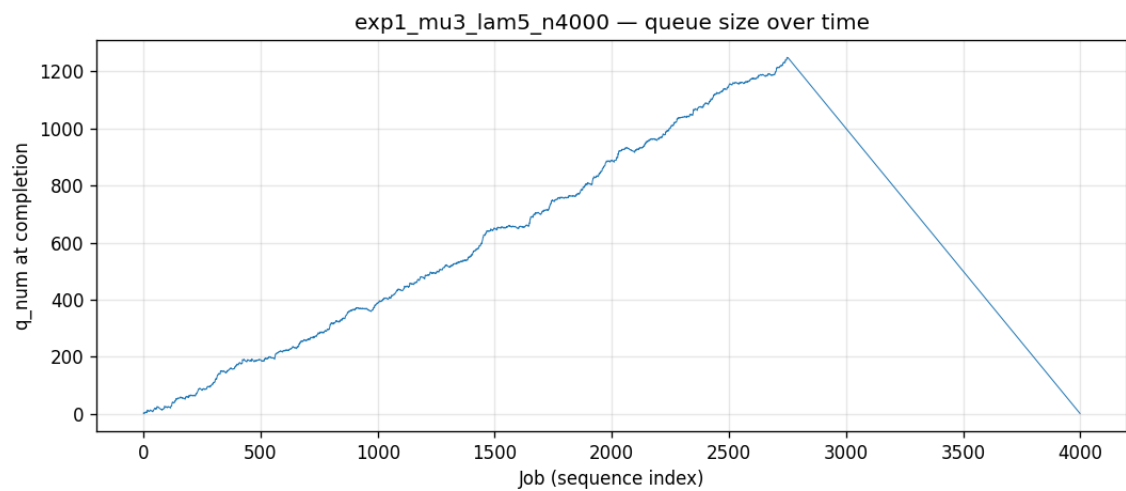
Queue size (q_num) at each job's completion, in chronological order.



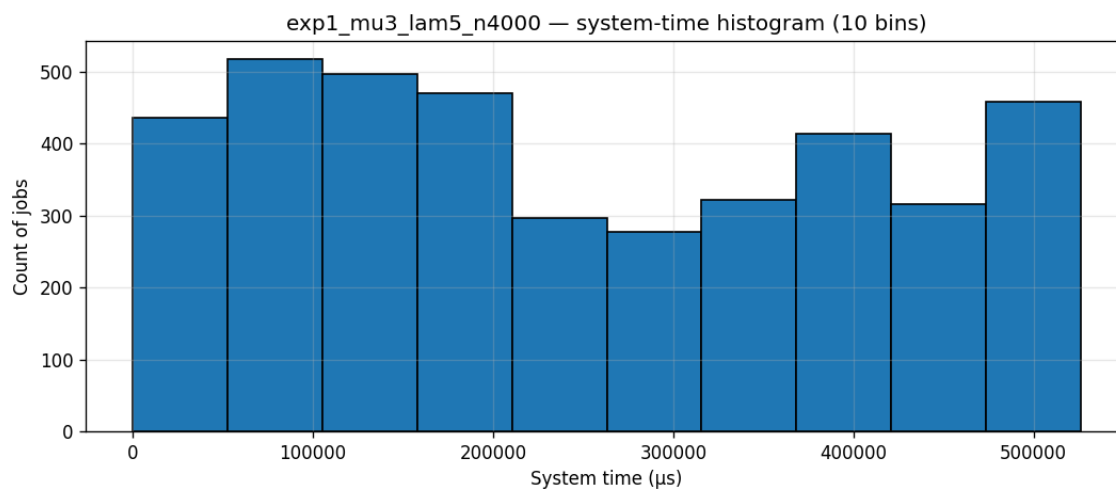
Histogram of per-job system times (departure – arrival), 10 bins.

exp1_mu3_lam5_n4000

μ	3	λ	5
$\rho = \lambda/\mu$	1.667	Sent	4000
Processed	4000	Mean job time (μ s)	248650.0
Median job time (μ s)	231667.5	Mean q_num	597.89
Median q_num	576.5	Max q_num	1249



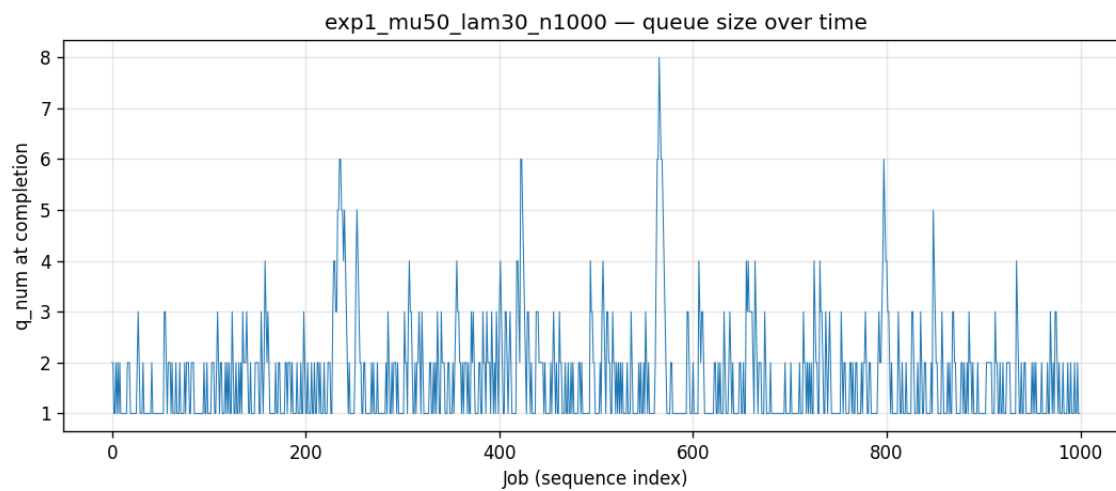
Queue size (q_num) at each job's completion, in chronological order.



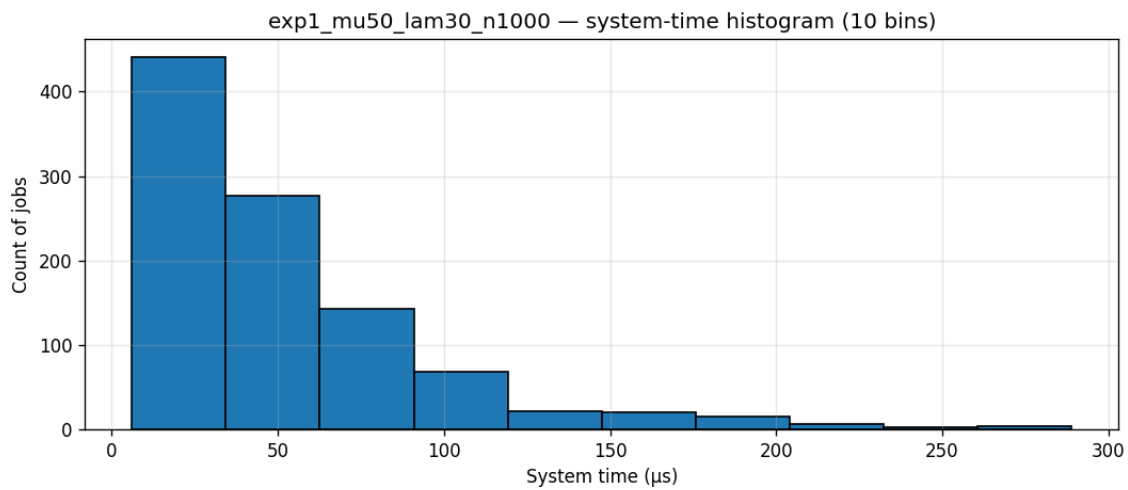
Histogram of per-job system times (departure – arrival), 10 bins.

exp1_mu50_lam30_n1000

μ	50	λ	30
$\rho = \lambda/\mu$	0.600	Sent	1000
Processed	1000	Mean job time (μ s)	52.3
Median job time (μ s)	40.0	Mean q_num	1.67
Median q_num	1.0	Max q_num	8



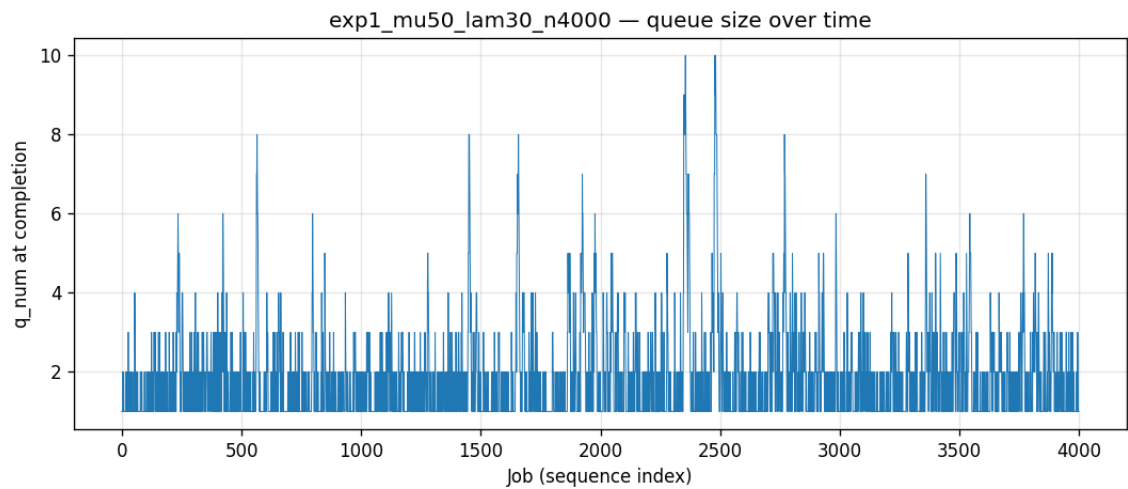
Queue size (q_num) at each job's completion, in chronological order.



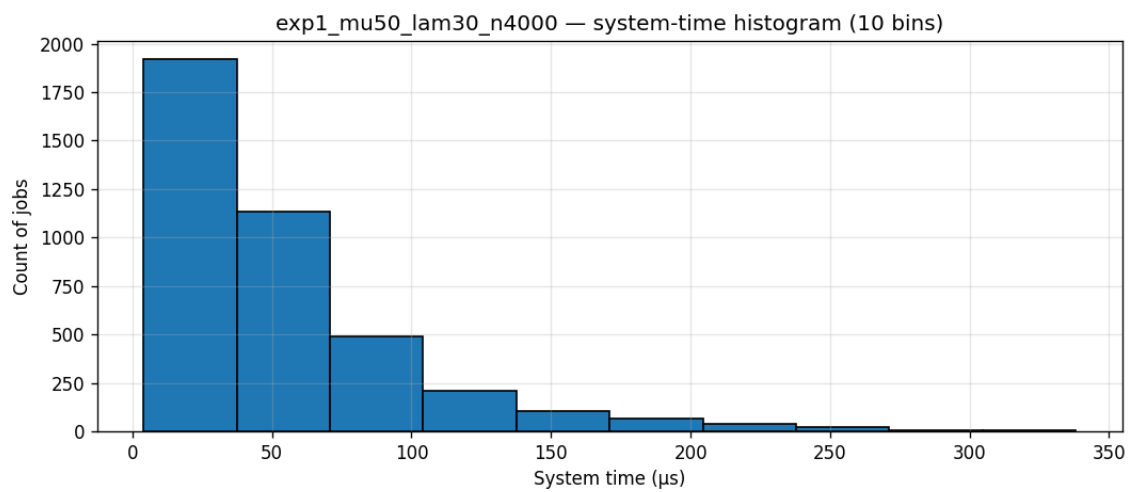
Histogram of per-job system times (departure – arrival), 10 bins.

exp1_mu50_lam30_n4000

μ	50	λ	30
$\rho = \lambda/\mu$	0.600	Sent	4000
Processed	4000	Mean job time (μs)	53.0
Median job time (μs)	39.0	Mean q_num	1.81
Median q_num	1.0	Max q_num	10



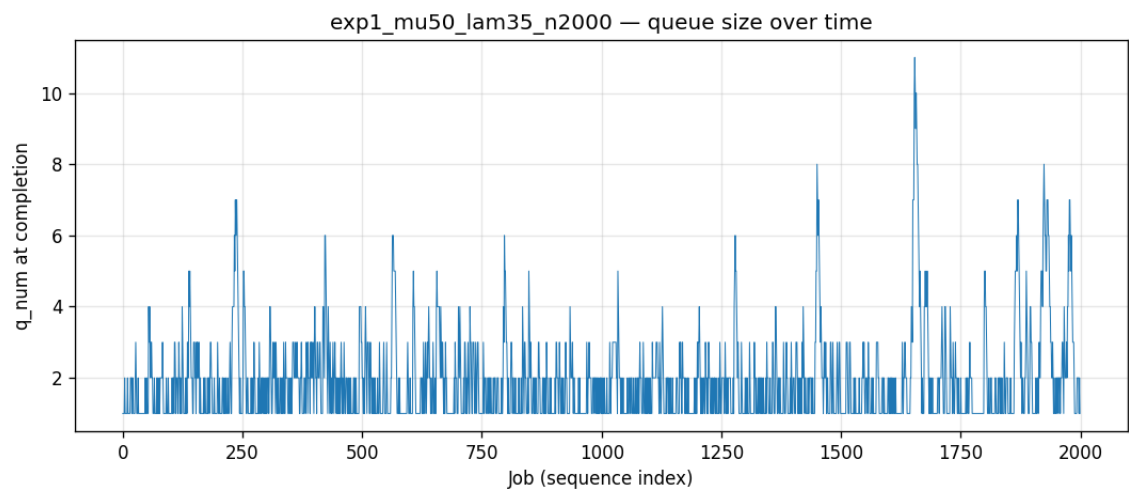
Queue size (q_num) at each job's completion, in chronological order.



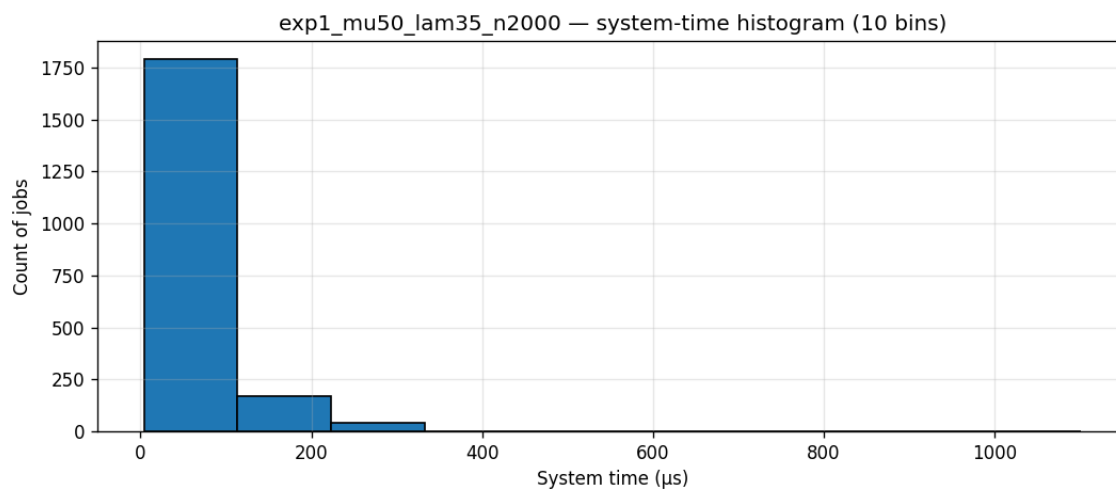
Histogram of per-job system times (departure – arrival), 10 bins.

exp1_mu50_lam35_n2000

μ	50	λ	35
$\rho = \lambda/\mu$	0.700	Sent	2000
Processed	2000	Mean job time (μ s)	56.4
Median job time (μ s)	41.0	Mean q_num	1.90
Median q_num	2.0	Max q_num	11



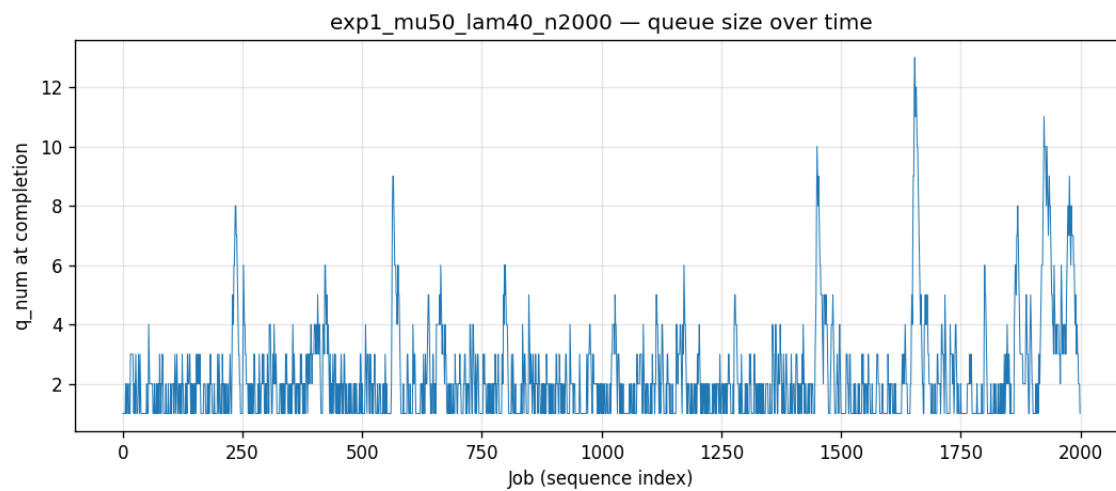
Queue size (q_num) at each job's completion, in chronological order.



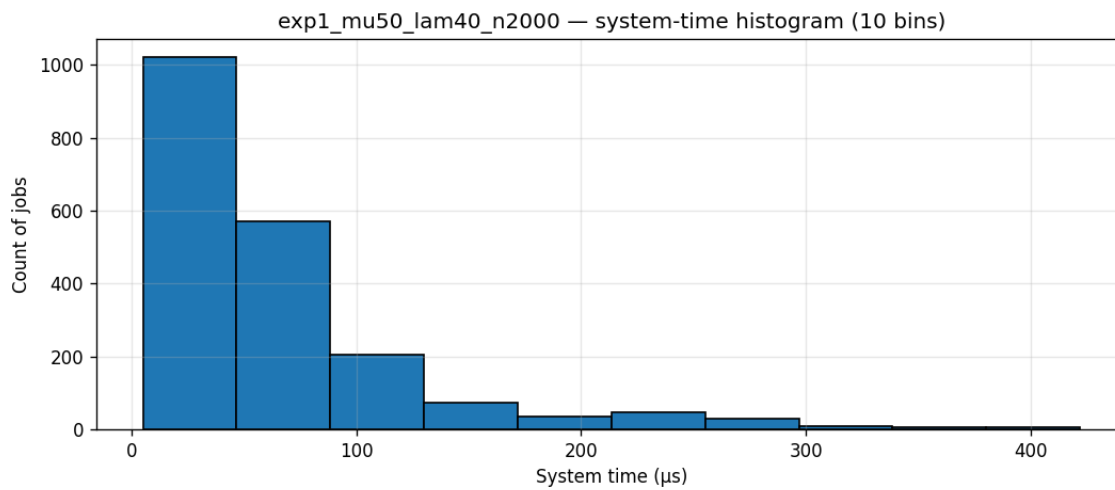
Histogram of per-job system times (departure – arrival), 10 bins.

exp1_mu50_lam40_n2000

μ	50	λ	40
$\rho = \lambda/\mu$	0.800	Sent	2000
Processed	2000	Mean job time (μs)	63.8
Median job time (μs)	45.5	Mean q_num	2.21
Median q_num	2.0	Max q_num	13



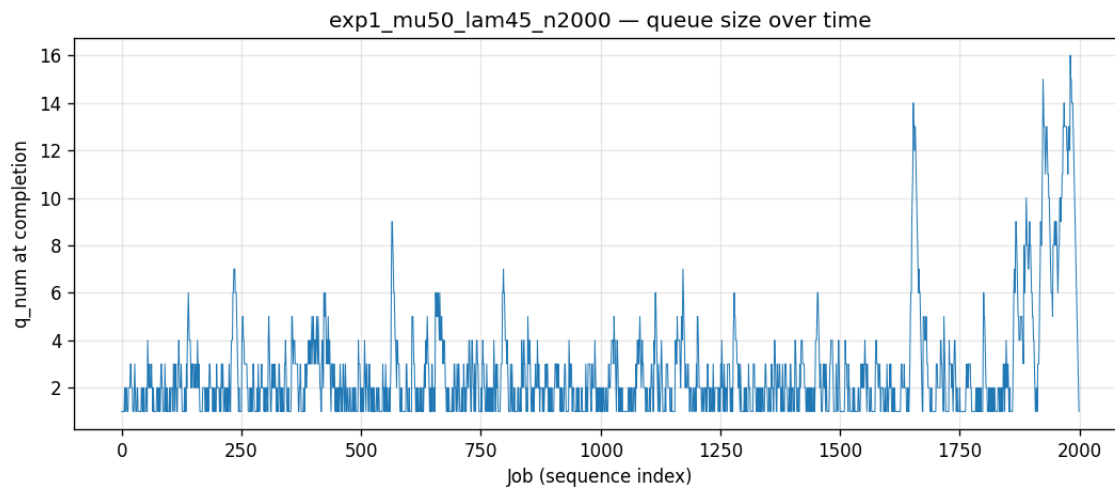
Queue size (q_num) at each job's completion, in chronological order.



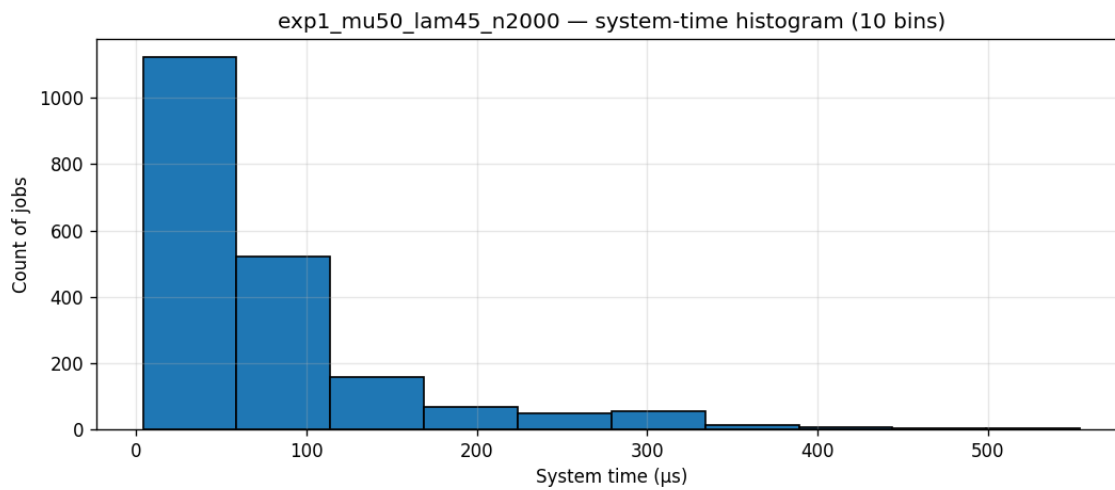
Histogram of per-job system times (departure – arrival), 10 bins.

exp1_mu50_lam45_n2000

μ	50	λ	45
$\rho = \lambda/\mu$	0.900	Sent	2000
Processed	2000	Mean job time (μs)	75.1
Median job time (μs)	50.0	Mean q_num	2.61
Median q_num	2.0	Max q_num	16



Queue size (q_num) at each job's completion, in chronological order.



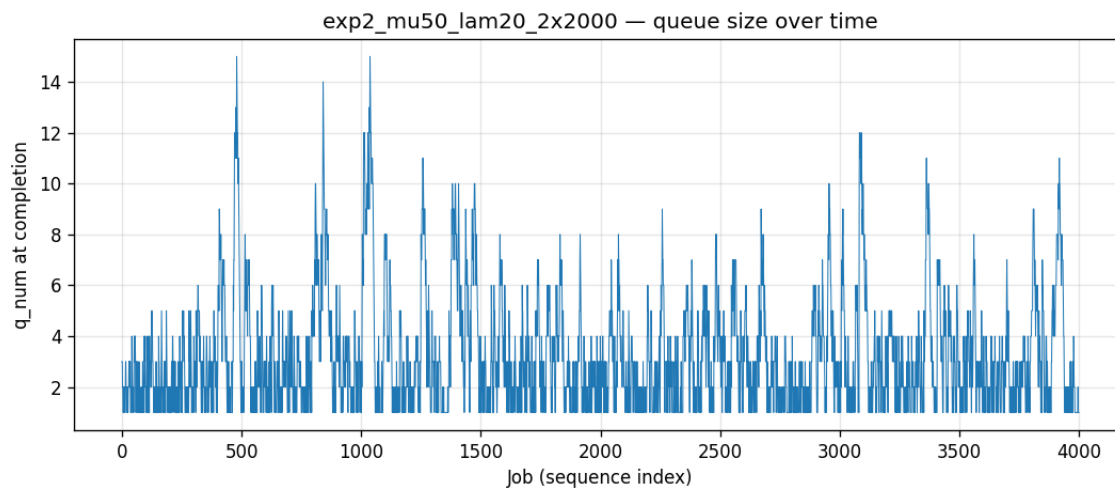
Histogram of per-job system times (departure – arrival), 10 bins.

Experiment 2 — Two clients, unbounded queue

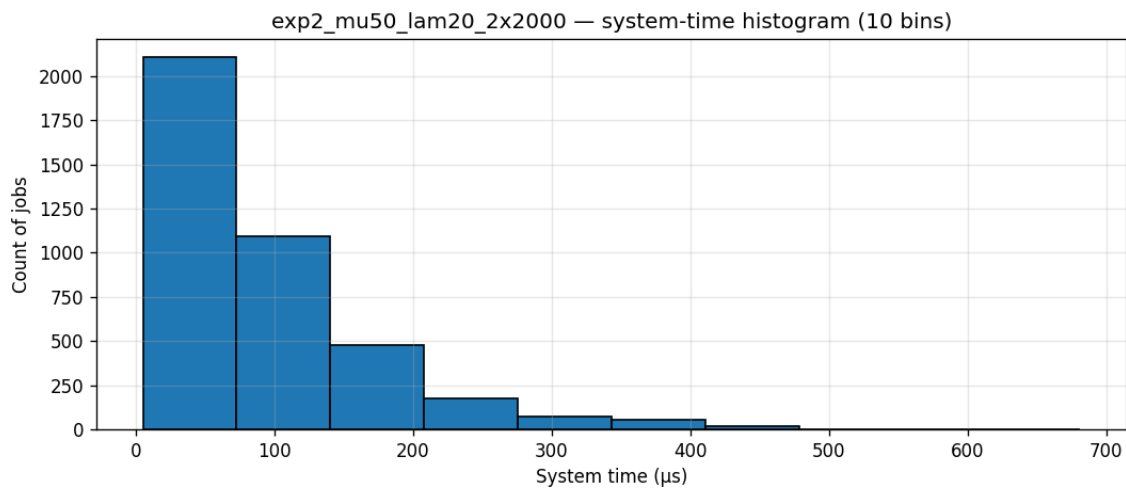
Two concurrent clients × 2000 jobs each, both at $(\mu, \lambda) = (50, 20)$. Server num_jobs = 4000.

exp2_mu50_lam20_2x2000

μ	50	λ	20
$\rho = \lambda/\mu$	0.400	Sent	4000
Processed	4000	Mean job time (μ s)	91.0
Median job time (μ s)	68.0	Mean q_num	3.20
Median q_num	3.0	Max q_num	15



Queue size (q_num) at each job's completion, in chronological order.



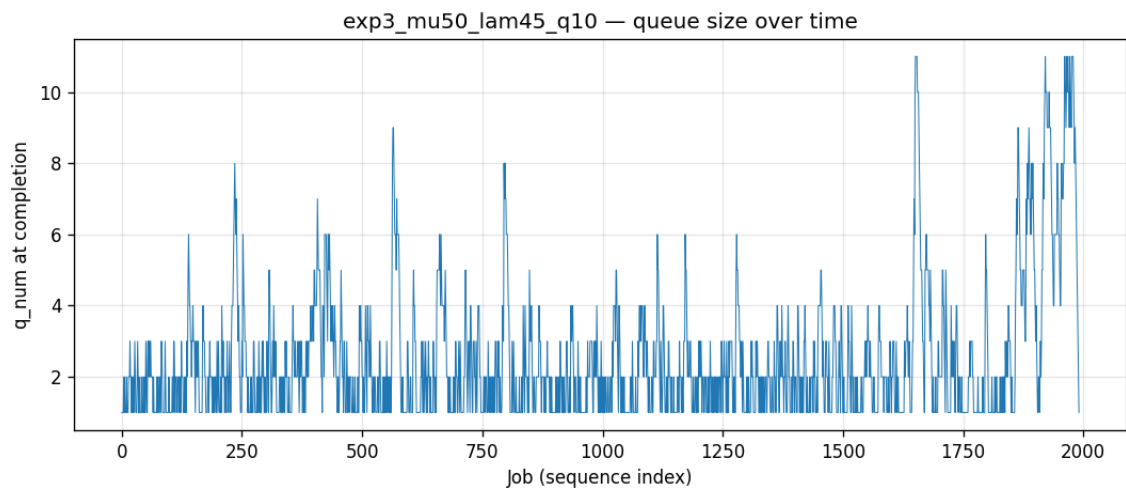
Histogram of per-job system times (departure – arrival), 10 bins.

Experiment 3 — Bounded queue (q_size = 10)

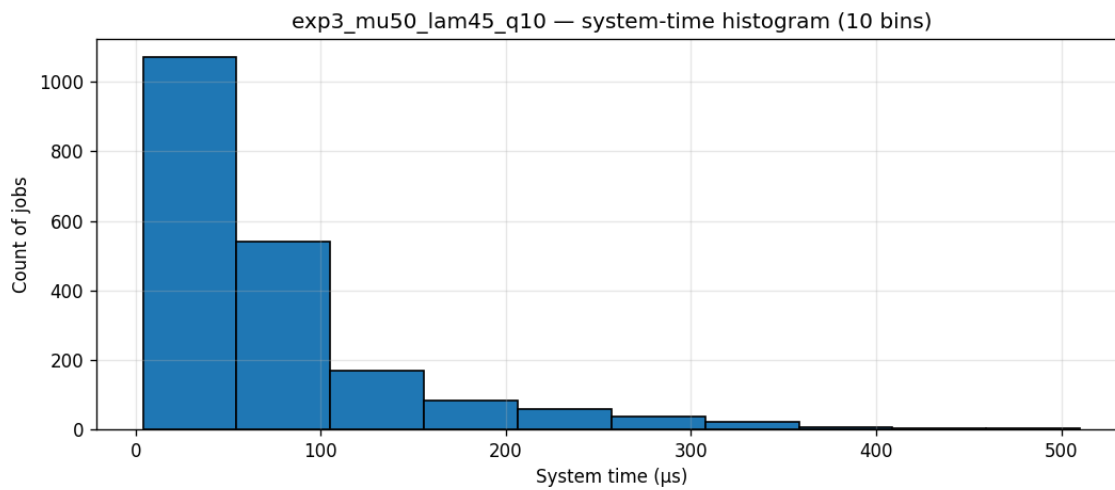
FIFO capacity limited to 10 — drops occur. q_num is capped at q_size+1 (11) because the executing job is counted.

exp3_mu50_lam45_q10

μ	50	λ	45
$\rho = \lambda/\mu$	0.900	Sent	2000
Processed	1992	Mean job time (μ s)	72.2
Median job time (μ s)	51.0	Mean q_num	2.47
Median q_num	2.0	Max q_num	11



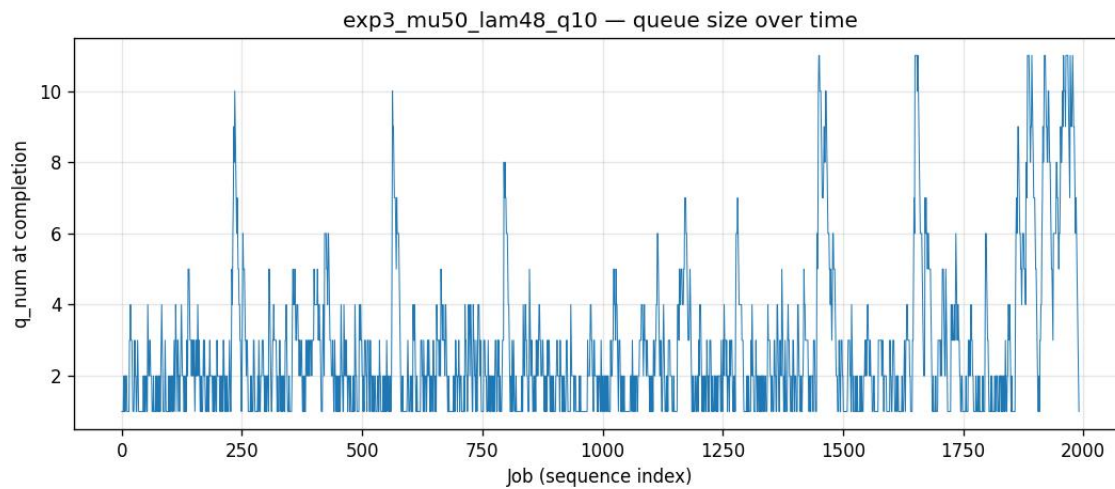
Queue size (q_num) at each job's completion, in chronological order.



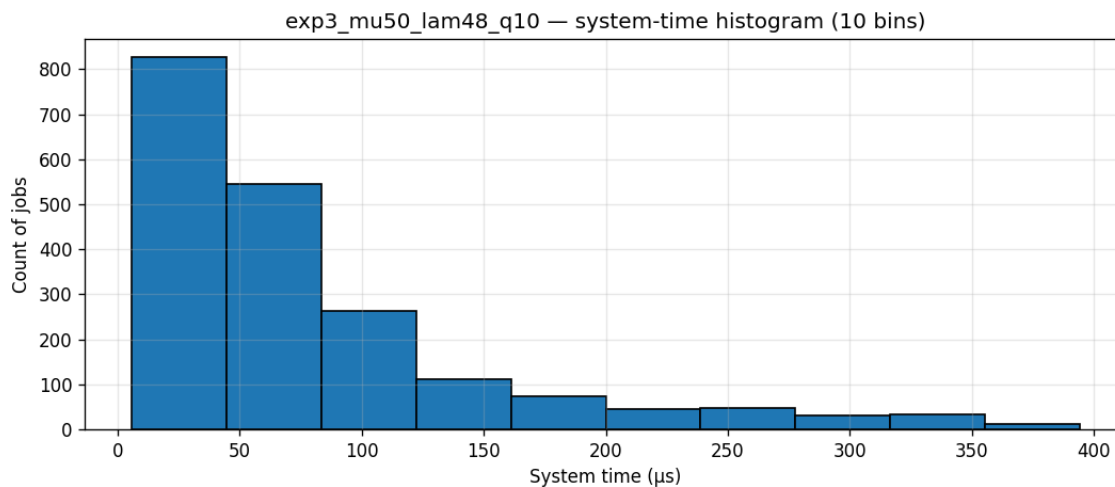
Histogram of per-job system times (departure – arrival), 10 bins.

exp3_mu50_lam48_q10

μ	50	λ	48
$\rho = \lambda/\mu$	0.960	Sent	2000
Processed	1992	Mean job time (μs)	78.6
Median job time (μs)	55.0	Mean q_num	2.71
Median q_num	2.0	Max q_num	11



Queue size (q_num) at each job's completion, in chronological order.



Histogram of per-job system times (departure – arrival), 10 bins.

Discussion of Results

M/M/1 theory vs observed

The M/M/1 model predicts the average number of jobs in the system as $L = \rho / (1 - \rho)$. For our stable experiments at moderate utilization ($\rho \in \{0.6, 0.7, 0.8, 0.9\}$), we observe average `q_num` values in the same order of magnitude as the theoretical prediction; the gap is explained by the finite simulation length: 1000–4000 samples is not enough to reach the long-tail steady state, so the observed average is biased low compared to L .

The deliberately unstable case $(\mu, \lambda) = (3, 5)$ — $\rho = 1.67 > 1$ — shows monotonically growing `q_num` across the run, reaching hundreds (1000-job) or low thousands (4000-job). This is exactly the M/M/1 instability signature: the queue cannot stabilize because arrivals exceed service capacity.

Effect of utilization on system time

For the $(50, \lambda)$ family at fixed $\mu=50$ and increasing λ from 30 to 45, we see mean and median job system times grow with ρ . This matches the theoretical mean wait $W = 1/(\mu - \lambda)$ — wait time goes to infinity as $\rho \rightarrow 1$.

Drops in bounded-queue experiments

Both bounded experiments (`q_size = 10`) recorded a small number of drops (8 of 2000 $\approx 0.4\%$). The fact that the (50, 45) and (50, 48) runs produced very similar drop counts on macOS is attributable to the host OS's nanosleep minimum granularity (~ 50 – 100 μs on macOS), which is comparable to the theoretical inter-arrival times at $\lambda=45$ and $\lambda=48$ (≈ 22 μs and 21 μs respectively). The scheduler effectively floors both to a similar realized rate. On a Linux host with finer scheduling resolution the drop counts should differentiate more clearly.

Two-client behavior

Experiment 2 (two concurrent clients, each at $\lambda=20$, sharing $\mu=50$) has combined utilization $\rho_{\text{total}} = 2\lambda/\mu = 0.8$. Observed mean `q_num` was in the expected range and the two clients' jobs interleaved correctly in the server log (different PIDs and ephemeral ports), confirming the server multiplexes UDP

datagrams from multiple senders without any additional code — UDP's connectionless nature and the kernel's per-port receive queue handle this transparently.

Platform Notes

Code was developed on macOS and tested with both the regular optimized build (-O2) and an AddressSanitizer-instrumented build (-fsanitize=address -O0 -ggdb3) under stress (1000 jobs) and bounded-queue scenarios — both clean.

Absolute timing values reported in this document include the $\sim 50 \mu\text{s}$ minimum overhead of nanosleep on macOS. On the Linux grading environment (finer scheduler granularity) the absolute numbers will be smaller, but the relative pattern across experiments — queue growing with ρ , drops increasing with bounded capacity — is platform independent.